

13. Eventual Consistency & Scaling Out: Google File System (GFS)

Michael Schöttner

Grundlagen Verteilter Systeme

SS 2024

13.1 Vorschau

- Verteilte Dateisysteme
- Google File System (GFS)



13.2 Verteilte Dateisysteme

- Unterstützen den Zugriff auf Dateien auf entfernten Servern
- Unterstützen in der Regel die Nebenläufigkeit:
 - Unterschiedliche Garantien, wer bei gleichzeitigen Schreibvorgängen "gewinnt"
- Implementieren Caching und Replikation sowie Fehlerbehandlung
- Verwenden meist lokale Dateisysteme für die Speicherverwaltung
- Viele Implementierungen vorhanden: für HPC, Cloud, Big-Data, ...



13.3 Google File System (GFS)

- Google benötigte ein verteiltes Dateisystem für seine große daten-intensive Anwendungen
- GFS ist die Basis für Google MapReduce, Google Bigtable, ...
 - Bigtable wird wiederum verwendet für YouTube, Google Analytics, Maps, ...
- Anforderung: redundante Speicherung riesiger Datenmengen auf kostengünstigen und "unzuverlässigen" Computern.



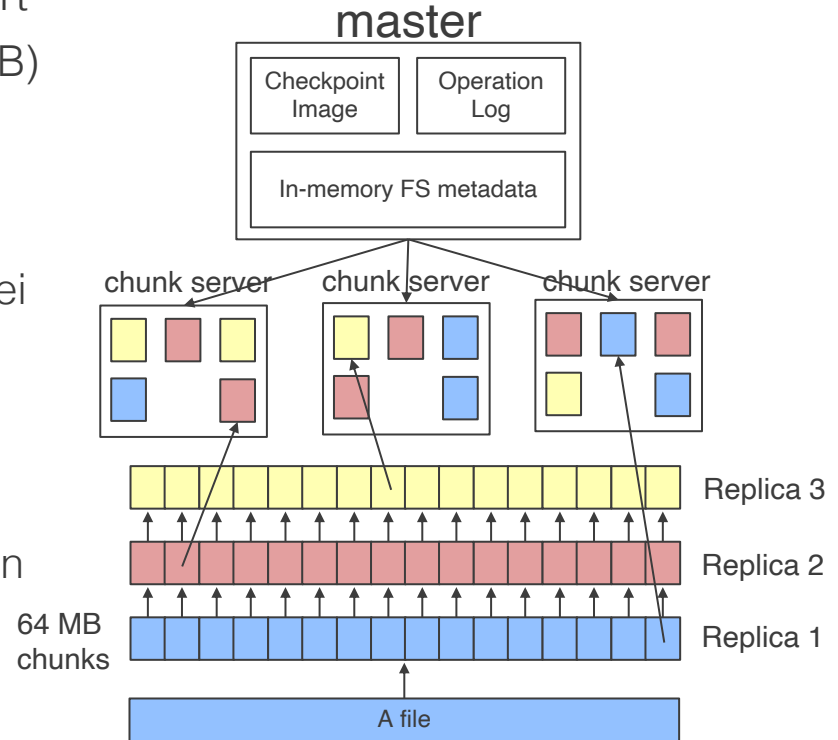
Zugriffsmuster der Google Apps

- „Moderate“ Anzahl an sehr großen Dateien
 - Millionen an Dateien
 - Die Dateien sind groß mind. 100 MB oder mehrere GB
- Dateien werden fast immer nur einmal beschrieben
 - Und dabei werden Daten i.d.R. am Ende einer Datei angehängt
 - Häufig hängen mehrere Clients gleichzeitig Daten in der gleichen Datei an
- In der Regel sequenzielle Lesezugriffe
- Wichtig ist ein hoher Durchsatz, statt einer kleinen Zugriffslatenz



Architektur

- Dateien werden als Chunks gespeichert
 - Chunks haben eine feste Größe (64 MB)
- Replikation
 - Jeder Chunk wird über mindestens drei Chunk-Server repliziert
- Kein Daten-Caching
 - Daten werden meistens einmal gelesen
 - Und Datensätze sind zu groß, um in Caches zu passen

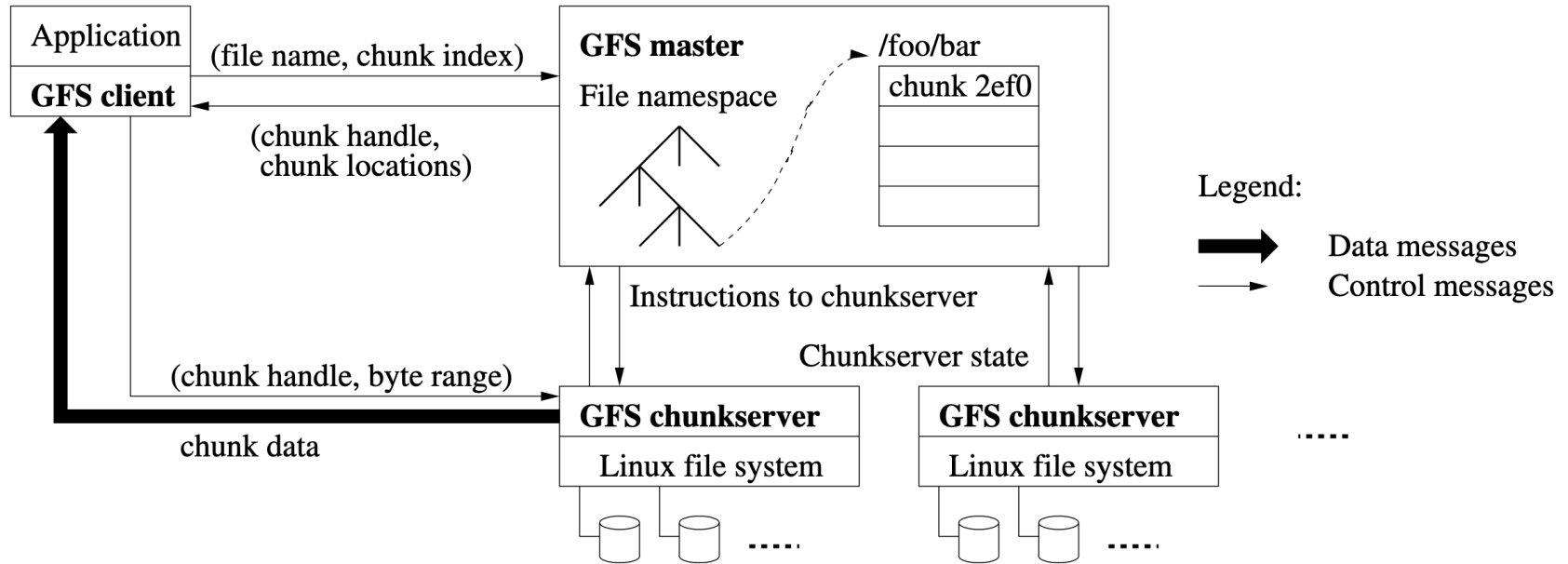


Architektur

- Ein Master
 - Koordiniert Zugriffe
 - Verwaltet Metadaten
- Viele Chunk-Server
 - In Racks gruppiert →
 - Verbunden durch Switches
 - Heartbeat-Nachrichten zur Erkennung von Ausfällen
- Mehrere Clients / Anwendungen



Architektur



- Verwaltet die Metadaten und den Namensraum
- Kommuniziert regelmäßig mit Chunk-Servern
 - Sammelt Infos bzgl. Auslastung & freiem Speicher
 - Erkennt Ausfälle, gibt Anweisungen
- Ist zuständig für die Erzeugung und (Re-)Replikation von Chunks
 - Verteilt Replikas über Racks und berücksichtigt Auslastung der Chunk-Server
 - Kann Chunks verschieben, um eine Lastverteilung durchzuführen

Master (2)

- Garbage Collection
 - Protokolliert das Löschen von Dateien und benennt diese vorerst um und „versteckt“ diese
 - Versteckte Dateien werden später, verzögert gelöscht
- Veraltete Replikate
 - Werden anhand von Chunk-Versionsnummern erkannt
 - Treten auf, falls ein Chunk-Server nach einem Crash erneut beitrifft



Metadaten

- Globale Metadaten werden auf dem Master gespeichert
 - Verzeichnisse und Dateinamen
 - Zuordnung von Dateien zu Chunks
 - Standorte der Replikate jedes Chunks
- Alle Metadaten werden im Arbeitsspeicher gehalten
 - Ermöglicht schnellen Zugriff
 - Dies ist möglich, weil pro Chunk nur 64 Byte Metadaten nötig sind
 - Und da Chunks groß sind, 64 MB, gibt es pro Datei nicht sehr viele Chunks

Example: metadata sizes

Cluster	A	B
Chunk Servers	342	227
Number of Files	735k	737k
Used Disk Space	55TB	155TB
Metadata at Master	48MB	60MB



Metadaten (2)

- Master speichert Operation-Log zur persistenten Protokollierung wichtiger Metadatenaktualisierungen.
 - Wird lokal auf Disk gespeichert sowie auf dem Schattenmaster repliziert
 - Periodisch werden Checkpoints gespeichert
 - Reduzieren die Log-Größe
 - Und ermöglichen schnelleres Recovery



Master - Fehlertoleranz

- Problem: Single-Point-Of-Failure
- Lösung: es werden Shadow-Master verwendet
- Problem: Bottleneck
- Lösung: GFS ist so gebaut, dass der Master nur selten kontaktiert wird
 - Master wird nur für Metadaten-Anfragen kontaktiert
 - Durch die großen Chunks ist dies nur selten notwendig
 - Zusätzlich werden die Metadaten auf den Clients gecacht
 - Durch die großen Chunks muss der Cache nicht sehr groß sein
 - Die Koordination von Schreibzugriffen wird auf die Chunk-Server verteilt, dort werden Primärkopien bestimmt (engl. primary replica)



Chunk-Server

- Speichert lokal 64 MB Chunks von GFS-Dateien
 - Jeder Chunk wird als eine Datei in einem Linux-Dateisystem gespeichert
 - Jeweils mit einer Versionsnummer und einer Prüfsumme
- Lese-/Schreibanfragen beinhalten ein Chunk-Handle sowie den Byte-Bereich innerhalb des Chunks
- Chunks werden repliziert, konfigurierbar, Default = 3
 - Rack-aware Replication: eine Kopie lokal, eine im selben Rack und eine in einem anderen Rack



Client

- Schickt Metadaten-Anfragen an den Master und cacht die Antworten
- Daten-Zugriffe erfolgen dann direkt über die Chunk-Server
- Daten werden auf der GFS-Ebene nicht gecacht
 - Dadurch werden Inkonsistenzen zwischen Caches vermieden
 - Zudem nutzt Caching nichts bei dem Zugriffsmuster der Google-Apps
 - Sequentielle Lesezugriffe (jeder Datensatz wird nur ein Mal gelesen)
 - Sequentielle Schreibzugriffe am Ende der Datei (keine Aktualisierungen)



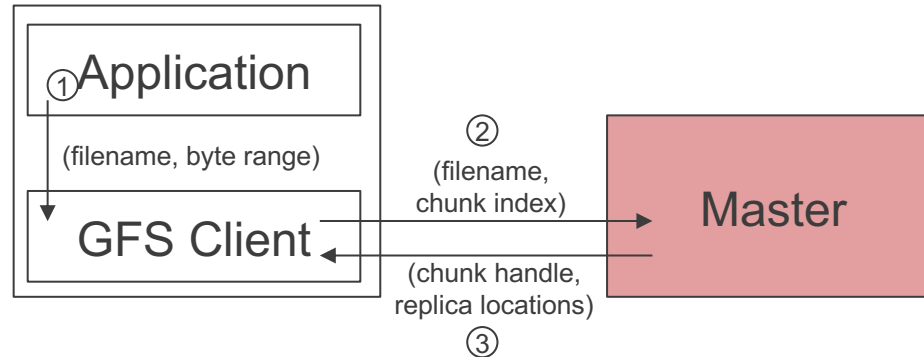
Client (2)

- Kein traditionelles Dateisystem:
 - Apps linken eine Bibliothek, über die sie dann auf GFS-Dateien zugreifen.
 - Chunks werden als Datei in einem darunterliegenden Linux-Dateisystem gespeichert
- API:
 - `create`, `delete`, `open`, `close`, `read`, `write`
 - `snapshot`: erzeuge eine Kopie von einer Datei
 - `append`: atomares Anhängen eines Records am Dateiende, mindestens ein Mal
 - Details siehe später



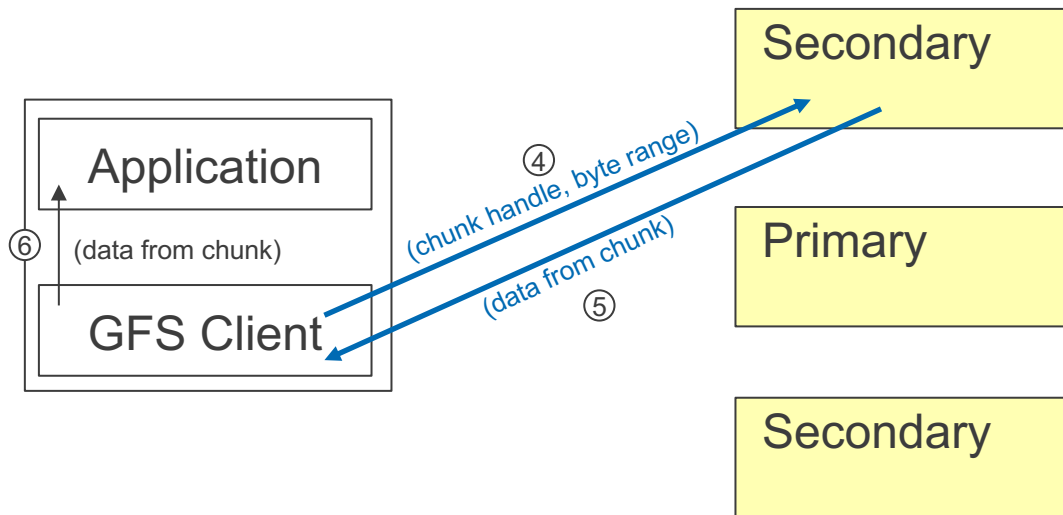
Lesen

1. Anwendung startet Leseanfrage
2. GFS-Client schickt Anfrage an den Master
3. Master antwortet mit einem Chunk-Handle und den Replikat-Orten



Lesen (2)

4. Client schickt dem nächsten¹ Replikat die Leseanfrage
5. Chunk-Server schickt die angefragten Daten zum Client
6. Client gibt die Daten an die Anwendung

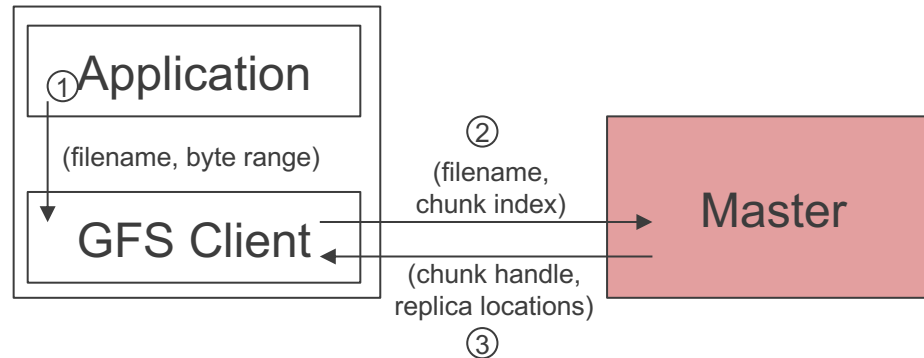


^{*)} Rack-Awareness



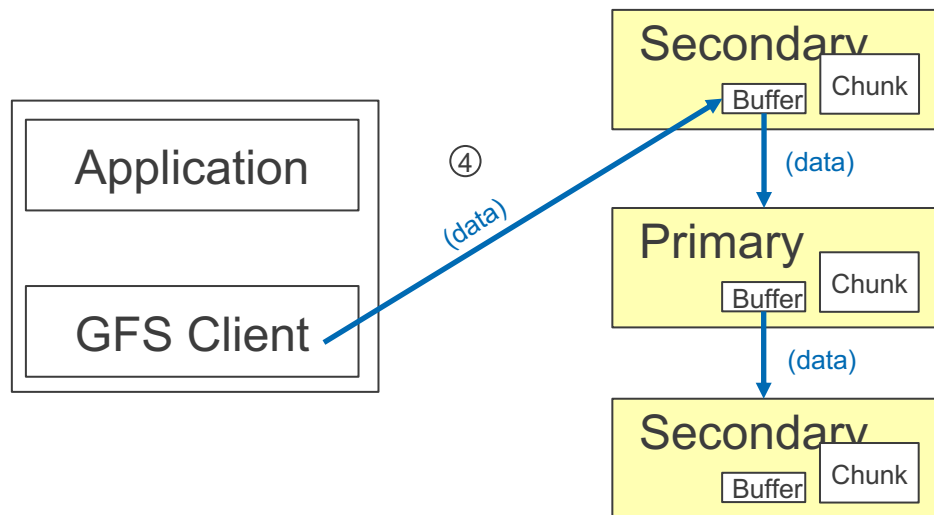
Schreiben

1. Anwendung startet Schreib Anfrage
2. GFS-Client schickt Anfrage an den Master
3. Master antwortet mit Chunk-Handle und den Replikat-Orten



Schreiben (2)

4. Client überträgt zu schreibende Daten an das nächste^{*)} Replikat
 - Daten werden gepuffert und in Batches an die Primärkopie übertragen und von dort aus an alle restlichen Replikate

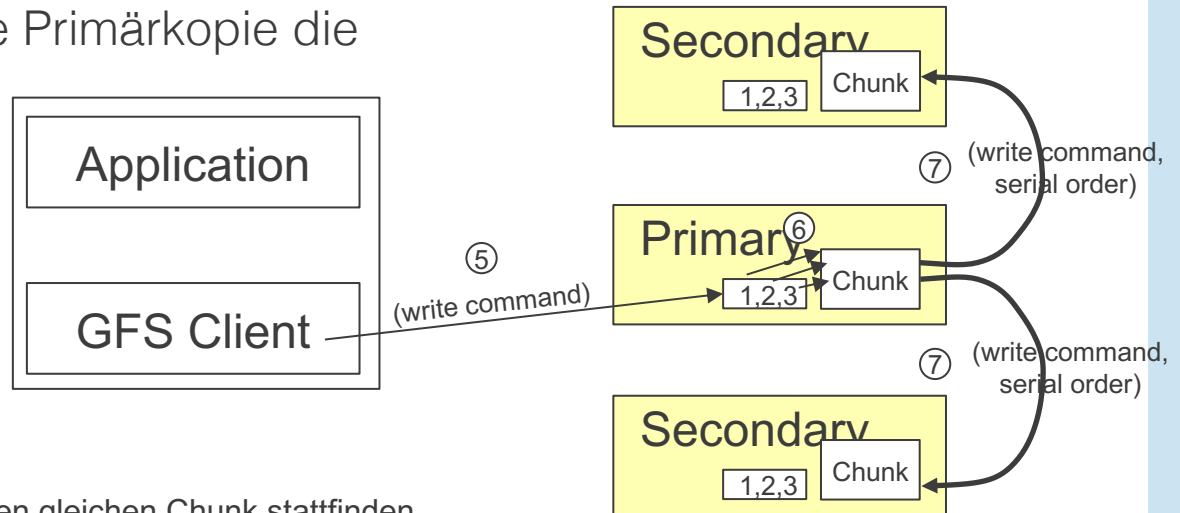


^{*)} Rack-Awareness



Schreiben (3)

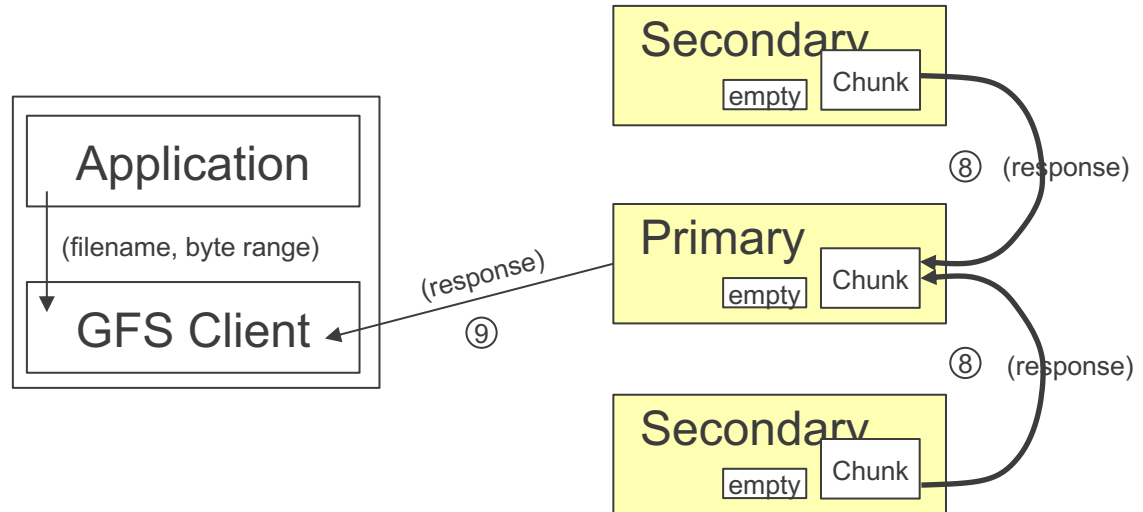
5. Danach schickt der Client den Schreib-Befehl an die Primärkopie, nachdem alle Daten übertragen wurden
6. Primärkopie bestimmt die serielle Ordnung von Updates^{*)} und schreibt diese in den lokalen Chunk
7. Anschließend fordert die Primärkopie die anderen Replikate auf, die Updates in der gleichen Reihenfolge zu schreiben



^{*)} es können mehrere Schreibzugriffe in den gleichen Chunk stattfinden

Schreiben (4)

8. Alle Replikate melden das erfolgreiche Schreiben zurück an die Primärkopie
9. Die Primärkopie informiert dann den Client



Atomic record append

- Dies ist die am häufigsten genutzte Operation in Google Apps
- Fügt einen Record am Ende einer Datei hinzu, atomar, mindestens ein Mal
 - Ist speziell für konkurrierende Schreibzugriffe in eine Datei gedacht
 - Z.B. Ergebnisse von Such-Bots in einer Datei sammeln



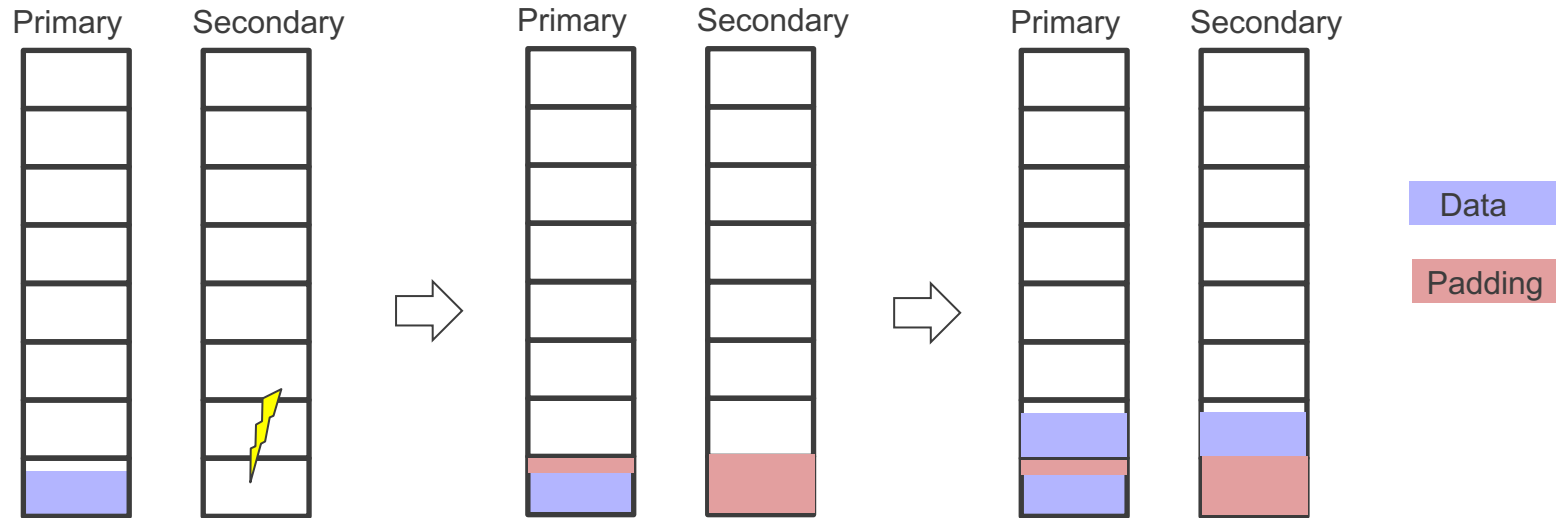
Atomic record append

- Ablauf wie beim Schreibzugriff, aber kein Offset vorgegeben
 1. Client schickt Daten zum nächsten Replikat (wie beim normalen Schreiben)
 2. Primärkopie prüft, ob die Daten in den letzten Chunk der Datei passen
 3. Falls ja:
 - Schreibt Record lokal und fordert die anderen Replikate auf, das gleiche zu tun, am gleichen Offset
 - Sammelt die Antworten und antwortet dann dem Client
 4. Falls nein:
 - Füllt den letzten Chunk mit Nullen auf und fordert die anderen Replikate auf, das gleiche zu tun
 - Informiert den Client, dass er die Operation mit dem nächsten Chunk wiederholen soll
 5. Falls die Operation bei einem Replikat fehlschlägt, oder die Daten nicht in den letzten Chunk passen, so wird die Operation durch den Client wiederholt



Atomic record append

- Durch das Wiederholen der Operation können Inkonsistenzen entstehen



Abgeschwächtes Konsistenzmodell

- Anwendungen müssen evt. Inkonsistenzen in den Dateien beachten und behandeln.
- Self-identifying records
 - Anwendungen speichern eine selbstgenerierte ID in ihren Records
 - Damit können sie einfach Duplikate beim Lesen erkennen und auslassen
- Self-validating chunks
 - Padding kann anhand von Checksums erkannt werden



Fehlertoleranz

- Master und Chunk-Server können in einigen Sekunden neu gestartet werden
- Chunks werden repliziert
- Master kann bei Bedarf durch Shadow-Master ersetzt werden



Zusammenfassung

- Google verwendet viele GFS-Instanzen
 - Jede mit hunderten/tausenden Chunk-Servern
 - Um Petabytes an Daten zu speichern
 - Bilden die Grundlage für andere Storage-Dienste, z.B. Bigtable (NoSQL)
- GFS ist ein skalierbares Dateisystem auf Commodity-Hardware
 - Spezielle für große Dateien
 - Einfache Master-Worker-Architektur
 - Schwache Konsistenz für Skalierbarkeit
 - GFS ist ein skalierbares Dateisystem auf Commodity-Hardware



Hadoop File System (HDFS)

- GFS ist in C geschrieben und closed source
- HDFS ist eine open source Nachimplementierung
 - In Java geschrieben
 - Damit plattformunabhängig
 - Speicher ebenfalls Chunks in einem darunterliegenden Dateisystem
- Erlaubt keine Änderungen in einer Datei und bietet auch keine parallelen Appends



- Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung, “The Google file system”, ACM symposium on Operating systems principles (SOSP), New York, USA, 2003.
- Konstantin Shvachko et. al., “The Hadoop Distributed File System”, IEEE/NASA Goddard Conference on Mass Storage Systems and Technologies, Washington, USA, 2010.